IBM Final Project

Github url:

https://github.com/Elvin-Chow/IBM-Data-Science-Capstone.git

Data collection and data wrangling methodology

```
# 导出 CSV
df.to_csv('spacex_web_scraped.csv', index=False)
```

```
  Flight No. Launch site                                Payload Payload
mass  \
0          1       CCAFS  Dragon Spacecraft Qualification Unit
0
1          2       CCAFS                                Dragon
0
2          3       CCAFS                                Dragon       52
5 kg
3          4       CCAFS                           SpaceX CRS-1     4,70
0 kg
4          5       CCAFS                           SpaceX CRS-2     4,87
7 kg

  Orbit Customer Launch outcome   Version Booster Booster landing  \
0   LEO   SpaceX      Success\n  F9 v1.07B0003.18         Failure
1   LEO     NASA        Success  F9 v1.07B0004.18         Failure
2   LEO     NASA        Success  F9 v1.07B0005.18    No attempt\n
3   LEO     NASA      Success\n  F9 v1.07B0006.18      No attempt
4   LEO     NASA      Success\n  F9 v1.07B0007.18    No attempt\n

              Date   Time
0      4 June 2010  18:45
1  8 December 2010  15:43
2      22 May 2012  07:44
3   8 October 2012  00:35
4     1 March 2013  15:10
```

We can now export it to a CSV for the next section, but to make the answers consistent and in case you have difficulties finishing this lab.

Following labs will be using a provided dataset to make each lab independent.

Data collection and data wrangling methodology

We can use the following line of code to determine the success rate:

```
[19]: df["Class"].mean()
```

```
[19]: np.float64(0.66666666666666666)
```

EDA with SQL results

Task 7

List the total number of successful and failure mission outcomes

[16]:
```
%%sql
SELECT Mission_Outcome, COUNT(*) as Total_Count
FROM SPACEXTABLE
GROUP BY Mission_Outcome;
```

```
 * sqlite:///my_data1.db
Done.
```

[16]:

Mission_Outcome	Total_Count
Failure (in flight)	1
Success	98
Success	1
Success (payload status unclear)	1

Task 8

List all the booster_versions that have carried the maximum payload mass, using a subquery with a suitable aggregate function.

[17]:
```
%%sql
SELECT DISTINCT Booster_Version
FROM SPACEXTABLE
WHERE PAYLOAD_MASS__KG_ = (SELECT MAX(PAYLOAD_MASS__KG_) FROM SPACEXTAB
```

```
 * sqlite:///my_data1.db
Done.
```

[17]:

Booster_Version
F9 B5 B1048.4
F9 B5 B1049.4
F9 B5 B1051.3
F9 B5 B1056.4
F9 B5 B1048.5
F9 B5 B1051.4
F9 B5 B1049.5
F9 B5 B1060.2
F9 B5 B1058.3
F9 B5 B1051.6
F9 B5 B1060.3
F9 B5 B1049.7

Task 9

List the records which will display the month names, failure landing_outcomes in drone ship ,booster versions, launch_site for the months in year 2015.

Note: SQLLite does not support monthnames. So you need to use substr(Date, 6,2) as month to get the months and substr(Date,0,5)='2015' for year.

[18]:
```
%%sql
SELECT substr(Date, 6, 2) as Month, Landing_Outcome, Booster_Version, L
FROM SPACEXTABLE
WHERE Landing_Outcome = 'Failure (drone ship)'
AND substr(Date, 1, 4) = '2015';
```

```
 * sqlite:///my_data1.db
Done.
```

[18]:

Month	Landing_Outcome	Booster_Version	Launch_Site
01	Failure (drone ship)	F9 v1.1 B1012	CCAFS LC-40
04	Failure (drone ship)	F9 v1.1 B1015	CCAFS LC-40

EDA with SQL results

Task 10

Rank the count of landing outcomes (such as Failure (drone ship) or Success (ground pad)) between the date 2010-06-04 and 2017-03-20, in descending order.

19]:
```
%%sql
SELECT Landing_Outcome, COUNT(*) as Outcome_Count
FROM SPACEXTABLE
WHERE Date BETWEEN '2010-06-04' AND '2017-03-20'
GROUP BY Landing_Outcome
ORDER BY Outcome_Count DESC;
```

```
 * sqlite:///my_data1.db
Done.
```

19]:

Landing_Outcome	Outcome_Count
No attempt	10
Success (drone ship)	5
Failure (drone ship)	5
Success (ground pad)	3
Controlled (ocean)	3
Uncontrolled (ocean)	2
Failure (parachute)	2
Precluded (drone ship)	1

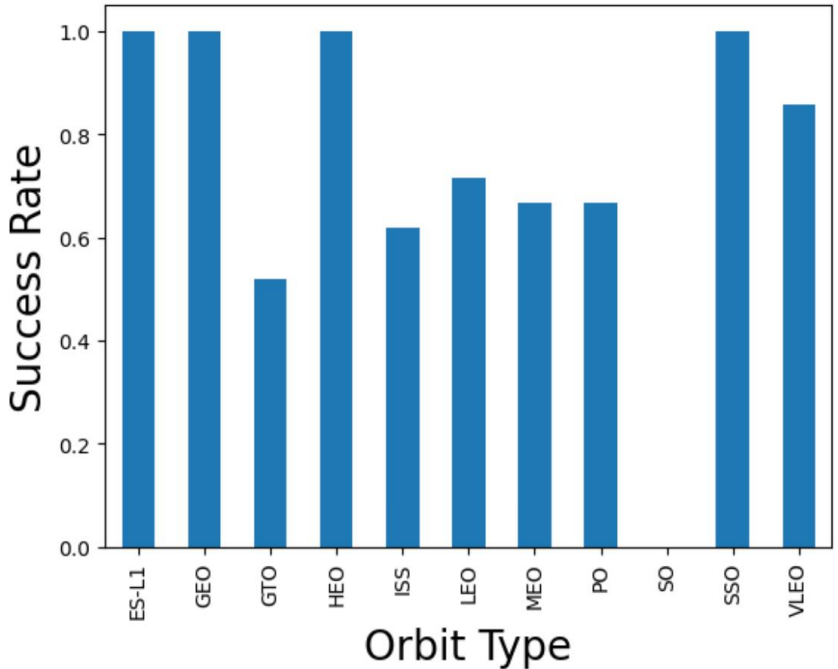
Data Visualization

TASK 3: Visualize the relationship between success rate of each orbit type

Next, we want to visually check if there are any relationship between success rate and orbit type.

Let's create a `bar chart` for the sucess rate of each orbit

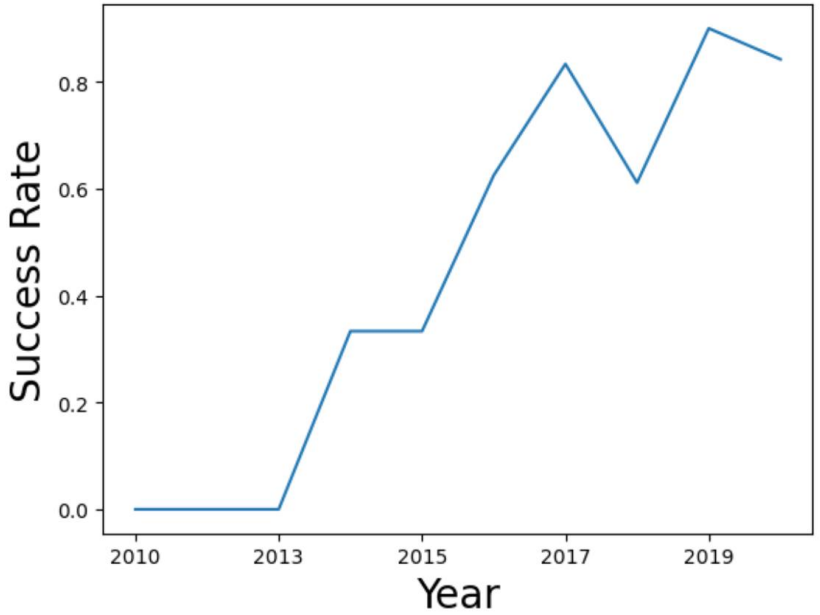
[7]:
```
# 使用 groupby 计算每个轨道的平均成功率并画图
df.groupby('Orbit')['Class'].mean().plot(kind='bar')
plt.xlabel("Orbit Type",fontsize=20)
plt.ylabel("Success Rate",fontsize=20)
plt.show()
```



The function will help you get the year from the date:

[15]:
```
def Extract_year():
    year = [] # 必须在函数内部初始化，确保每次调用都是空的
    for i in df["Date"]:
        year.append(i.split("-")[0])
    return year
```

[16]:
```
# 提取年份并计算每年的平均成功率
df_copy = df.copy()
df_copy['Year'] = Extract_year()
df_copy.groupby('Year')['Class'].mean().plot(kind='line')
plt.xlabel("Year",fontsize=20)
plt.ylabel("Success Rate",fontsize=20)
plt.show()
```



Data Visualization

TASK 7: Create dummy variables to categorical columns

Use the function `get_dummies` and `features` dataframe to apply OneHotEncoder to the column `Orbits`, `LaunchSite`, `LandingPad`, and `Serial`. Assign the value to the variable `features_one_hot`, display the results using the method head. Your result dataframe must include all features including the encoded ones.

```
# HINT: Use get_dummies() function on the categorical columns
# 选择需要转换的特征列
features = df[['FlightNumber', 'PayloadMass', 'Orbit', 'LaunchSite',

# 使用 get_dummies 进行独热编码
features_one_hot = pd.get_dummies(features, columns=['Orbit', 'Launch
features_one_hot.head()
```

	FlightNumber	PayloadMass	Flights	GridFins	Reused	Legs	Block	Reuse
0	1	6104.959412	1	False	False	False	1.0	
1	2	525.000000	1	False	False	False	1.0	
2	3	677.000000	1	False	False	False	1.0	
3	4	500.000000	1	False	False	False	1.0	
4	5	3170.000000	1	False	False	False	1.0	

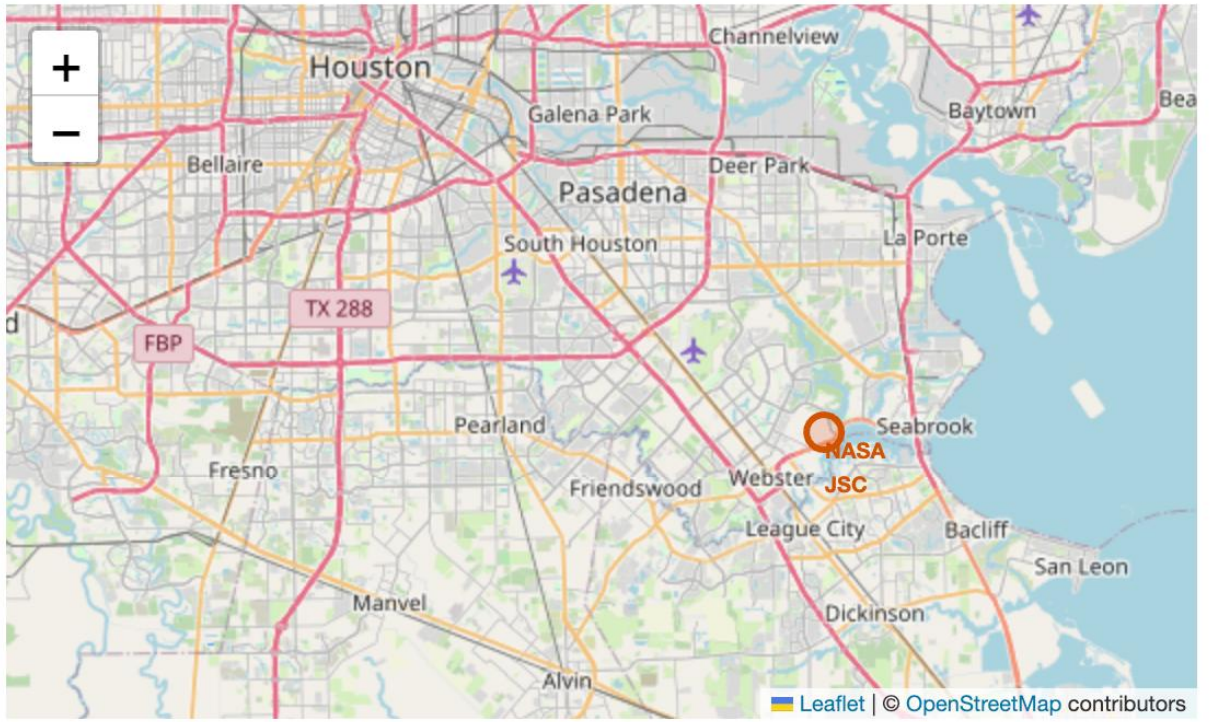
5 rows × 80 columns

TASK 8: Cast all numeric columns to `float64`

Now that our `features_one_hot` dataframe only contains numbers, cast the entire dataframe to variable type `float64`

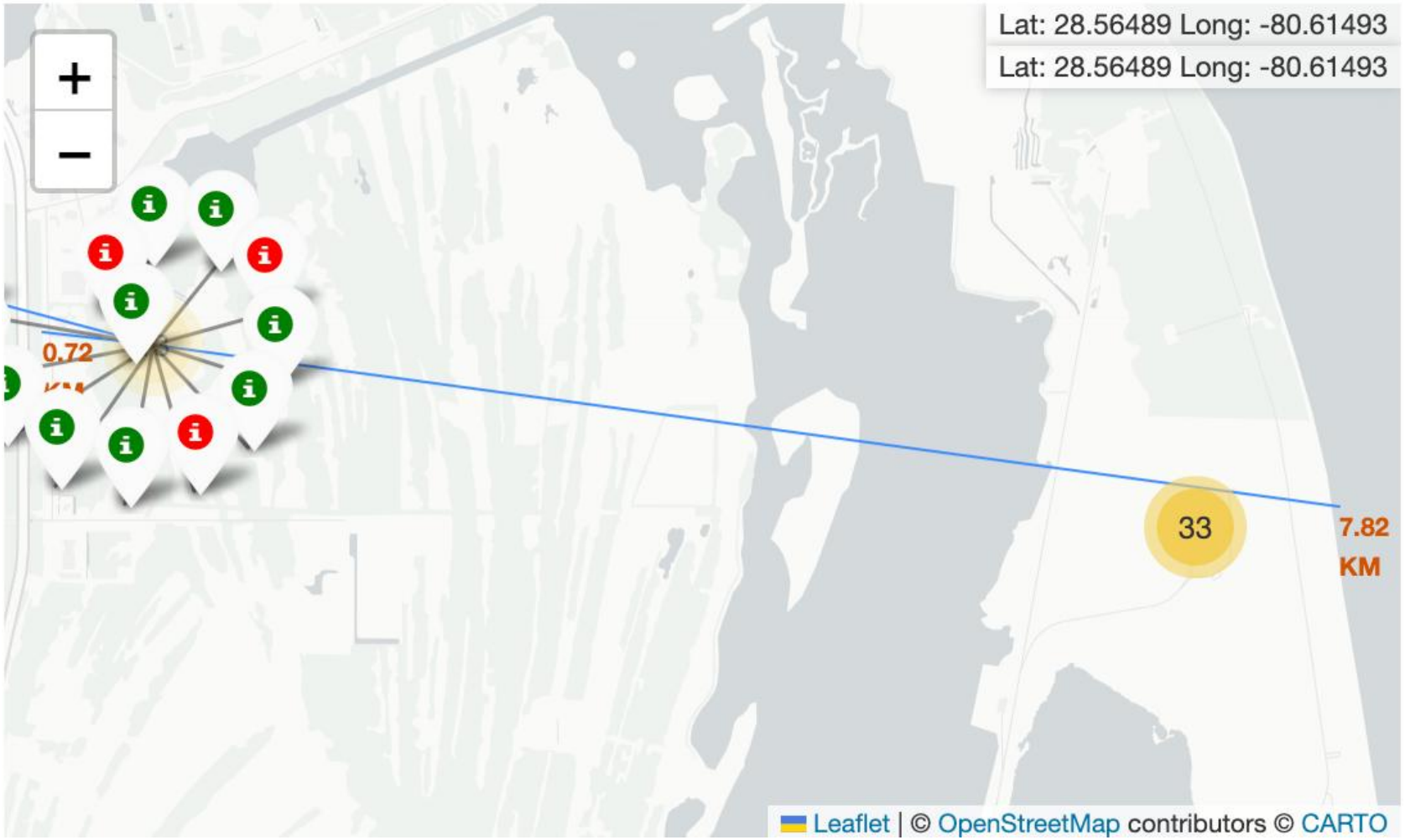
```
# HINT: use astype
# 转换数据类型以满足后续模型要求
features_one_hot = features_one_hot.astype('float64')
```

Interactive Visual Analytics with Folium

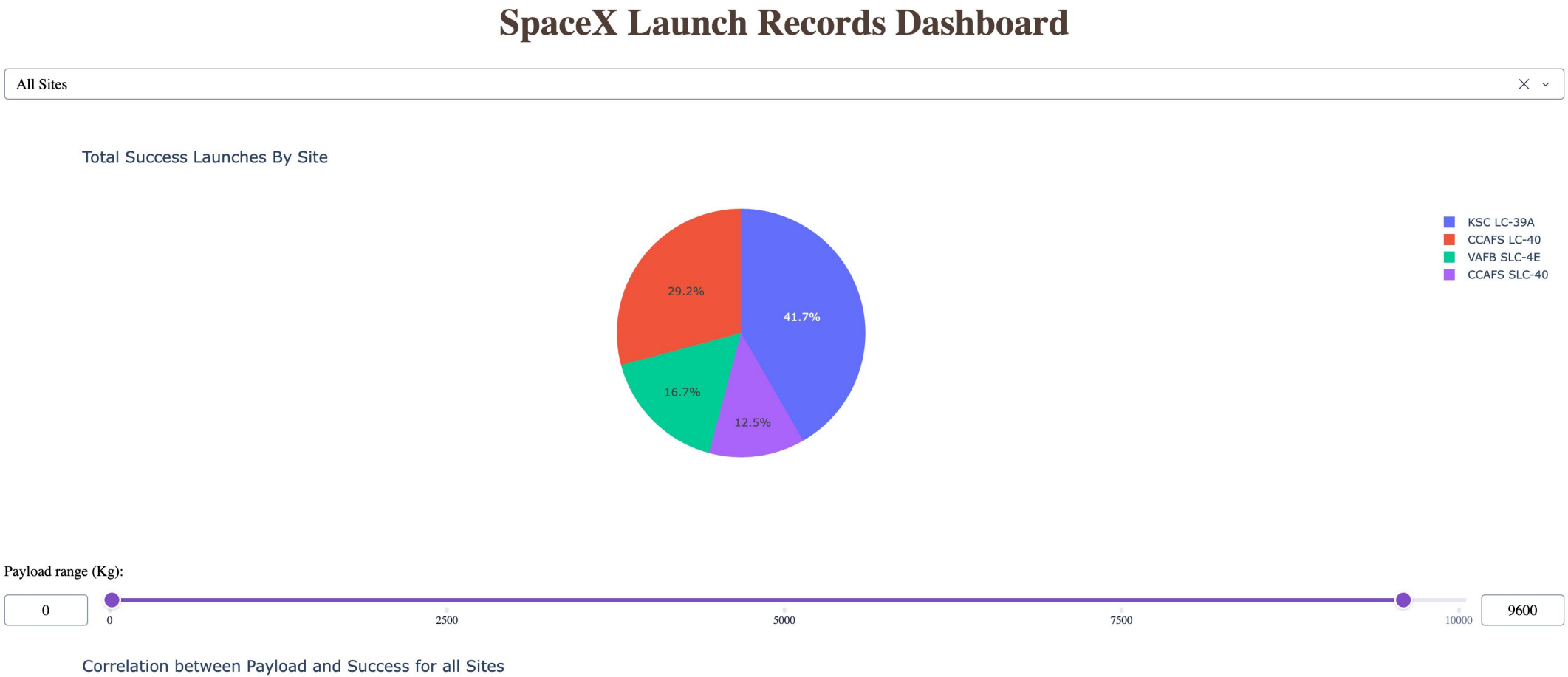


Interactive Visual Analytics with Folium

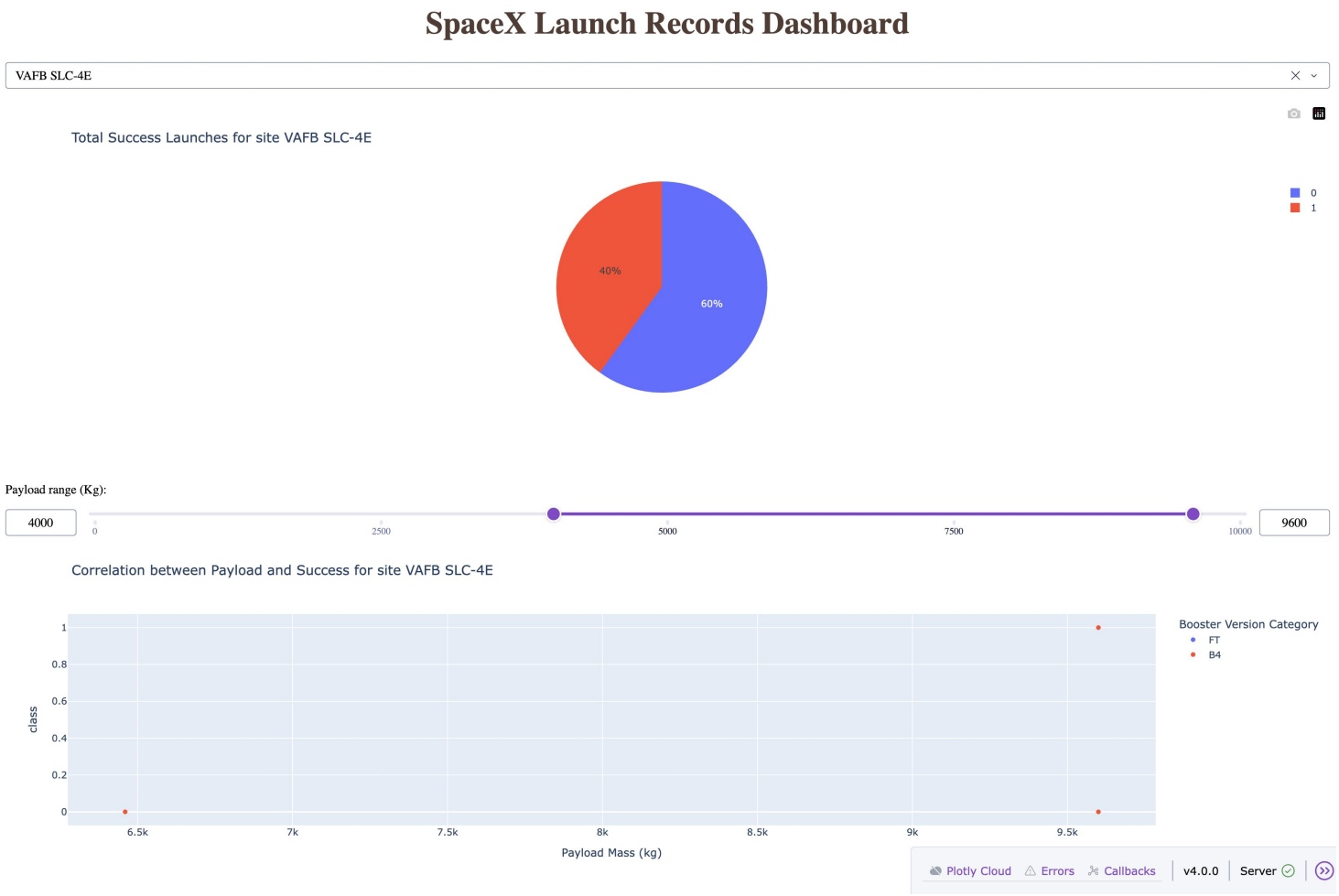
Interactive Visual Analytics with Folium



Dash


SpaceX Launch Records Dashboard
All Sites
Total Success Launches By Site
29.2%
41.7%
16.7%
12.5%
KSC LC-39A
CCAFS LC-40
VAFB SLC-4E
CCAFS SLC-40
Payload range (Kg):
0
0
2500
5000
7500
10000
9600
Correlation between Payload and Success for all Sites

DASH


SpaceX Launch Records Dashboard
VAFB SLC-4E
Total Success Launches for site VAFB SLC-4E
0
1
40%
60%
Payload range (Kg):
4000
0
2500
5000
7500
10000
9600
Correlation between Payload and Success for site VAFB SLC-4E
class
1
0.8
0.6
0.4
0.2
0
6.5k
7k
7.5k
8k
8.5k
9k
9.5k
Payload Mass (kg)
Booster Version Category
FT
B4
Plotly Cloud
Errors
Callbacks
v4.0.0
Server

Predictive analysis (classification) results

TASK 4

Create a logistic regression object then create a GridSearchCV object `logreg_cv` with cv = 10. Fit the object to find the best parameters from the dictionary `parameters`.

[16]:
```
lr = LogisticRegression()
logreg_cv = GridSearchCV(lr, parameters, cv=10)
logreg_cv.fit(X_train, Y_train)
parameters ={'C':[0.01,0.1,1],
             'penalty':['l2'],
             'solver':['lbfgs']}
```

[17]:
```
parameters ={"C":[0.01,0.1,1],'penalty':['l2'], 'solver':['lbfgs']}# l1
lr=LogisticRegression()
```

We output the `GridSearchCV` object for logistic regression. We display the best parameters using the data attribute `best_params_` and the accuracy on the validation data using the data attribute `best_score_`.

[18]:
```
print("tuned hpyerparameters :(best parameters) ",logreg_cv.best_params
print("accuracy :",logreg_cv.best_score_)
```

```
tuned hpyerparameters :(best parameters)  {'C': 0.01, 'penalty': 'l2',
'solver': 'lbfgs'}
accuracy : 0.8464285714285713
```

TASK 5

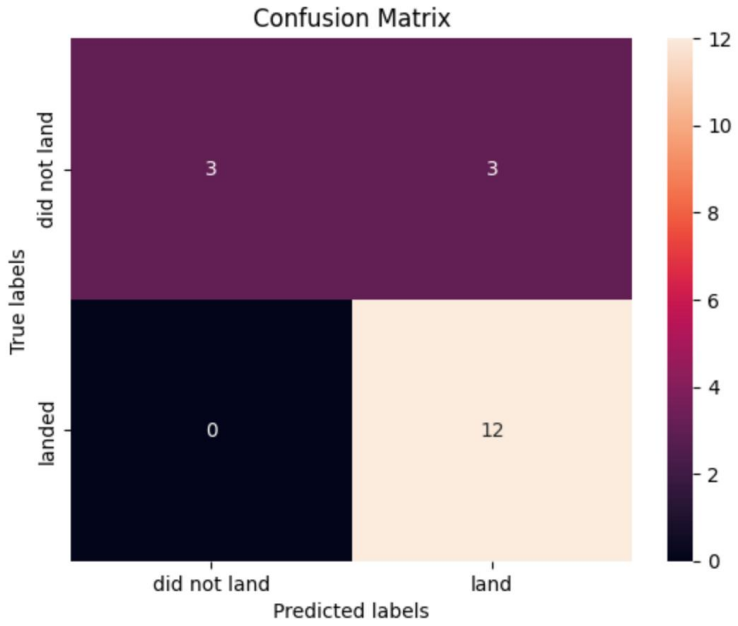
Calculate the accuracy on the test data using the method `score`:

[19]:
```
# 在测试集上计算准确率
logreg_score = logreg_cv.score(X_test, Y_test)
print("Test set accuracy: ", logreg_score)
```

```
Test set accuracy:  0.8333333333333334
```

Lets look at the confusion matrix:

[20]:
```
yhat=logreg_cv.predict(X_test)
plot_confusion_matrix(Y_test,yhat)
```



TASK 6

Create a support vector machine object then create a `GridSearchCV` object `svm_cv` with cv = 10. Fit the object to find the best parameters from the dictionary `parameters`.

[21]:
```
parameters = {'kernel':('linear', 'rbf','poly','rbf', 'sigmoid'),
              'C': np.logspace(-3, 3, 5),
              'gamma':np.logspace(-3, 3, 5)}
svm = SVC()
```

[22]:
```
svm_cv = GridSearchCV(svm, parameters, cv=10)
svm_cv.fit(X_train, Y_train)
```

[22]:
```
▸ GridSearchCV
  ▸ estimator: SVC
    ▸ SVC
```

[23]:
```
print("tuned hpyerparameters :(best parameters) ",svm_cv.best_params_)
print("accuracy :",svm_cv.best_score_)
```

```
tuned hpyerparameters :(best parameters)  {'C': 1.0, 'gamma': 0.0316227
7660168379, 'kernel': 'sigmoid'}
accuracy : 0.8482142857142856
```

TASK 7

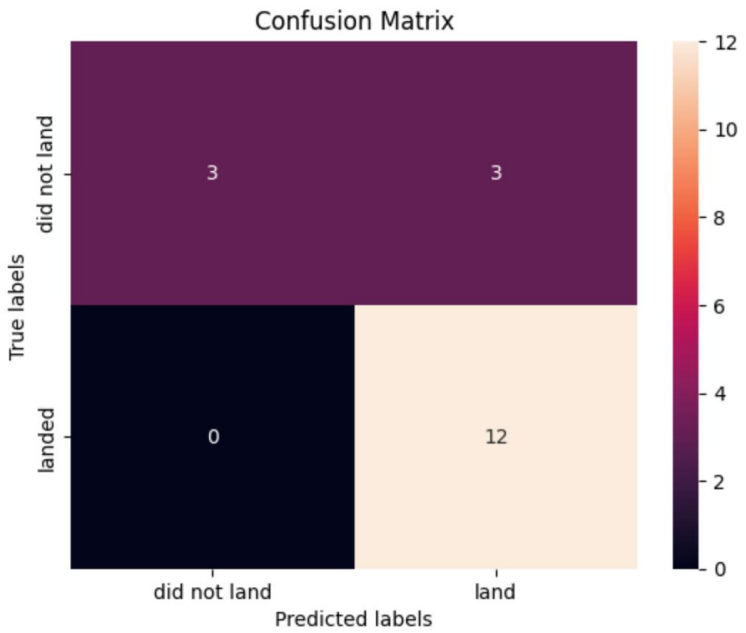
Calculate the accuracy on the test data using the method `score`:

[24]:
```
svm_score = svm_cv.score(X_test, Y_test)
print("Test set accuracy: ", svm_score)
```

```
Test set accuracy:  0.8333333333333334
```

We can plot the confusion matrix

[25]:
```
yhat=svm_cv.predict(X_test)
plot_confusion_matrix(Y_test,yhat)
```



TASK 8

Create a decision tree classifier object then create a `GridSearchCV` object `tree_cv` with cv = 10. Fit the object to find the best parameters from the dictionary `parameters`.

[42]:
```
# 修改后的参数字典，去掉了已经失效的 'auto'
parameters = {'criterion': ['gini', 'entropy'],
     'splitter': ['best', 'random'],
     'max_depth': [2*n for n in range(1,10)],
     'max_features': ['sqrt'], # 删掉 'auto'，只保留 'sqrt'
     'min_samples_leaf': [1, 2, 4],
     'min_samples_split': [2, 5, 10]}

tree = DecisionTreeClassifier()
```

[43]:
```
tree_cv = GridSearchCV(tree, parameters, cv=10)
tree_cv.fit(X_train, Y_train)
```

[43]:
```
▸ GridSearchCV
▾ estimator: DecisionTreeClassifier
DecisionTreeClassifier()
  ▸ DecisionTreeClassifier
```

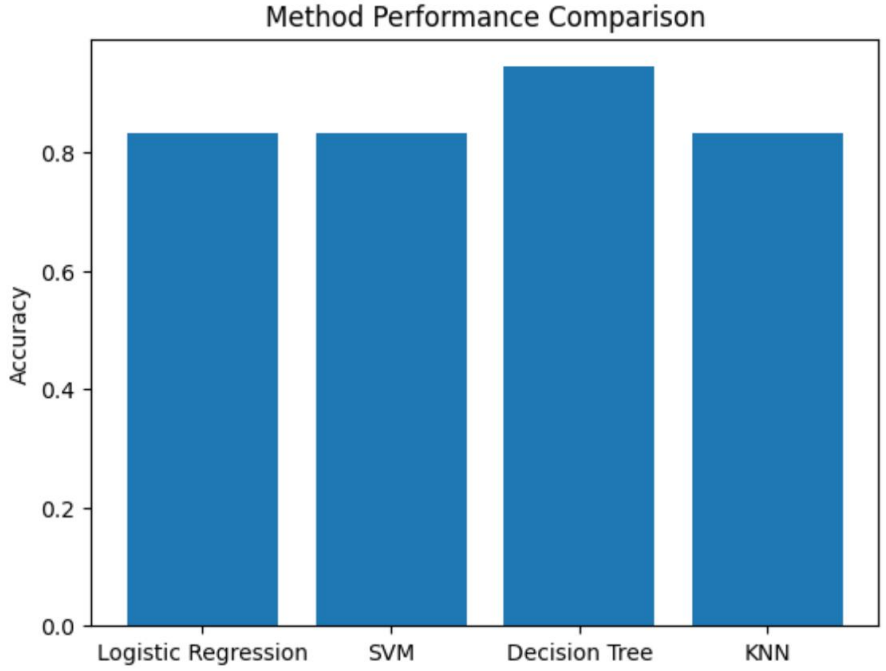
[44]:
```
print("tuned hpyerparameters :(best parameters) ",tree_cv.best_params_)
print("accuracy :",tree_cv.best_score_)
```

```
tuned hpyerparameters :(best parameters)  {'criterion': 'gini', 'max_de
pth': 8, 'max_features': 'sqrt', 'min_samples_leaf': 1, 'min_samples_sp
lit': 5, 'splitter': 'random'}
accuracy : 0.875
```

Find the method performs best:

[55]:
```
algorithms = {'Logistic Regression': logreg_score, 'SVM': svm_score, 'D

best_algorithm = max(algorithms, key=algorithms.get)
print('Best algorithm is \"', best_algorithm, '\" with accuracy of', al

# 可选：画个柱状图来对比（对 PPT 很有帮助）
import matplotlib.pyplot as plt
plt.bar(algorithms.keys(), algorithms.values())
plt.ylabel('Accuracy')
plt.title('Method Performance Comparison')
plt.show()
```

```
Best algorithm is " Decision Tree " with accuracy of 0.9444444444444444
```